Problem E. 提取权重

“由于 Emily 很忙”... 我咋感觉 Emily 挺闲的。

—烤鸡块君

这是一道交互题。

Emily 有一棵包含 n 个节点的树，树上编号为 i 的节点的权值是 w_i 。1 号节点是根节点，且根节点的权值为 0 。

Emily 希望你猜出每个节点的权值。具体来说，你可以提出至多 n 个询问，第 i 个询问包含两个整数 u_i 和 v_i。如果 u_i 到 v_i 的简单路径包含恰好 k 条边，Emily 会告诉你节点 u_i 到节点 v_i 的简单路径上所有节点的权值按位异或 † 的结果（包括端点）。否则，Emily 会回答你 “-1”，表示她不想回答这个问题。Emily 很忙，因此只有在你提出了所有的问题后，她才会开始回答。

当然，对于某些情况，你可能无法通过不超过 n 个询问猜出所有点的权值。此时，你不应该发出任何询问，而是直接告诉 Emily 这是不可能做到的。

† 按位异或是指将两个二进制数字的每一位进行模 2 意义下的加法运算。例如：$(0011)_2 \oplus (0101)_2 = (0110)_2$ 。

Input

每个测试文件仅有一组测试数据。

第一行包含两个整数 n 和 k $(2 \le n \le 250, 1 \le k \le n-1)$ ，表示树的节点个数以及询问中提及的参数。

接下来 $n-1$ 行，每行包含两个整数 x_i 和 y_i $(1 \le x_i, y_i \le n)$ ，表示节点 x_i 和 y_i 之间连有一条边。

保证所有的边构成一棵树，且正确答案满足 $0 \le w_i < 2^{30}$ 。

Interaction Protocol

首先，你需要判断能否在 n 个询问内猜出所有节点的权值。如果不能，你的程序应该输出 “`No`” 并立刻退出。否则，你的程序应该输出 “`Yes`”，然后开始询问。你可以以任意的大小写形式输出你的答案。例如，“`yEs`”，“`yes`”，“`Yes`” 和 “`YES`” 都将被视为肯定的回复。

要提出询问，请以 “`?` q u_1 v_1 u_2 v_2 ... u_q v_q ” $(1 \le q \le n$ 和 $1 \le u_i, v_i \le n)$ 的格式**一次**输出你的**所有** q 个询问。在清空输出缓冲区后，你的程序需要读入一行 q 个整数。第 i 个整数表示第 i 个询问的回答。

要猜想权值，请以 “`!` w_2 w_3 ... w_n” $(0 \le w_i < 2^{30})$ 的格式输出你的猜测结果。在清空输出缓冲区之后，您的程序应该立即退出。

请注意，每组测试数据的答案都是预先确定的。也就是说，裁判程序并不是适应性的。还请注意，输出猜想不算一次询问。

清空输出缓冲区可以使用以下方式：

- C 和 C++ 使用 `fflush(stdout)` （如果您使用 `printf`）或 `cout.flush()` （如果您使用 `cout`）。
- Java 和 Kotlin 使用 `System.out.flush()` 。
- Python 使用 `stdout.flush()` 。

Examples

standard input	standard output
4 1 1 2 2 3 2 4 1 3 2	 YES ? 3 1 2 2 3 2 4 ! 1 2 3
5 2 1 2 2 3 3 4 3 5 1 2 3 4	 YES ? 4 1 3 2 4 2 5 4 5 ! 4 5 3 2
6 2 1 2 2 3 3 4 4 5 4 6	 NO